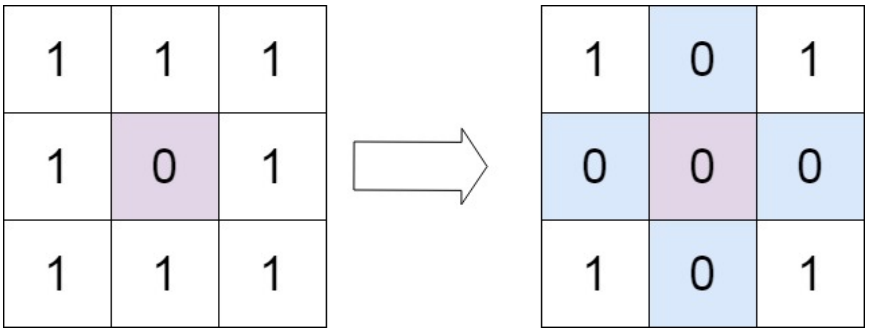
Matrix

leetcode

73.矩阵置零

- 一个矩阵，如果有一个元素为0，则将其所在行列的元素均设为0。



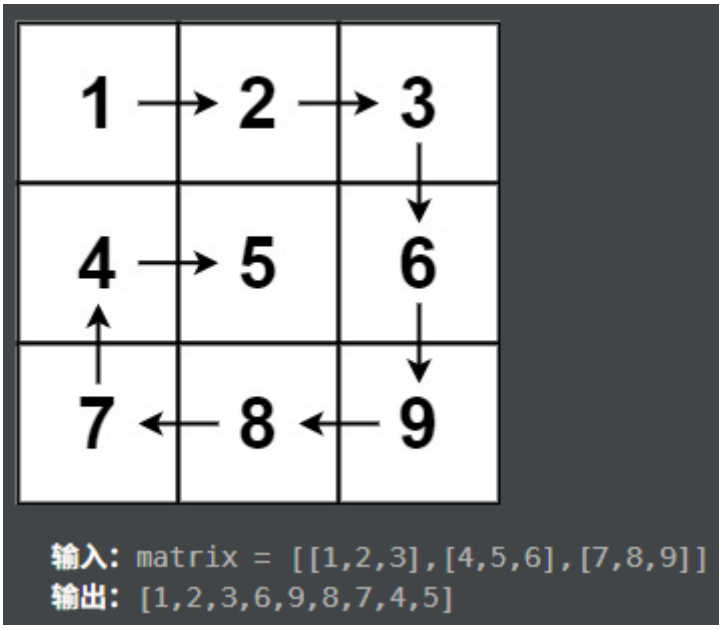
- 思路：第一遍遍历矩阵，标记出现0元素的行和列。第二遍遍历矩阵，将标记的行列元素设为0。

```
class Solution {
public:
    void setZeroes(vector<vector<int>>& matrix) {
        int rowLen = matrix.size();
        int colLen = matrix[0].size();
        vector<bool> row(rowLen),col(colLen); // 标记出现0的行和列
        // 遍历矩阵，标记出现0的行和列
        for(int i=0;i<rowLen;i++){
            for(int j=0;j<colLen;j++){
                if( matrix[i][j]==0 ){
                    row[i] = true;
                    col[j] = true;
                }
            }
        }
        // 遍历矩阵，将在出现0的行和列的元素设为0
        for(int i=0;i<rowLen;i++){
            for(int j=0;j<colLen;j++){
                if( row[i]||col[j]){
                    matrix[i][j] = 0;
                }
            }
        }
    }
};
```

-

54.螺旋矩阵

- 将矩阵按照 **顺时针螺旋顺序**，返回元素。



- 思路：利用“模拟法”，通过一个**方向数组**控制遍历路径，并在遇到边界或已访问节点时自动“右转”。

这个算法模拟了一个机器人在矩阵中行走的过程：

1. 初始化：从左上角 `(0,0)` 开始，初始方向为“向右”。
2. 循环遍历：总共需要走 `rowLen * colLen` 步。
3. 记录与标记：
   - 将当前位置的值存入结果数组 `ans`。
   - 将当前位置标记为 `INT_MAX`（表示“已访问”或“墙”），防止重复访问。
4. 试探下一步：
   - 根据当前方向 `di`，计算下一个理论位置 `(x, y)`。
   - 判断合法性：检查 `(x, y)` 是否越界（超出矩阵范围）或者是否已经访问过（值为 `INT_MAX`）。
5. 决策转向：
   - 如果不合法：说明前方是墙或边界，需要顺时针旋转90度（`di = (di + 1) % 4`）。
   - 如果合法：保持原方向。
6. 移动：更新当前坐标 `(i, j)` 为新的方向上的下一步。

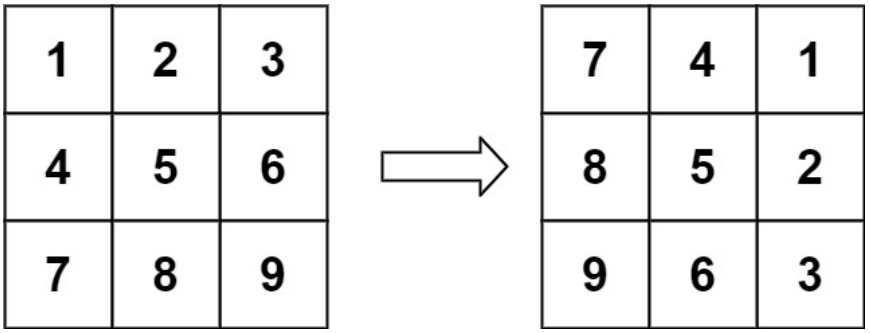
```
class Solution {
    int directions[4][2] = {{0,1},{1,0},{0,-1},{-1,0}}; // 方向向量-右下左上，逆时针方向
public:
    vector<int> spiralOrder(vector<vector<int>>& matrix) {
        int rowLen = matrix.size(),colLen = matrix[0].size();
```

```
        vector<int> ans(rowLen*colLen);
        int i=0,j=0; // 当前遍历的矩阵元素
        int di = 0; // 当前的前进方向
        for(int k=0; k<rowLen*colLen; k++){
            ans[k] = matrix[i][j];
            matrix[i][j] = INT_MAX;
            // 计算下一个位置，判断是否需要进行转向
            int x = i + directions[di][0]; // 行位置
            int y = j + directions[di][1]; // 列位置
            if( x<0 || x>=rowLen || y<0 || y>=colLen || matrix[x][y]==INT_MAX
){
                di = (di+1)%4; // 右转90°
            }
            i = i + directions[di][0];
            j = j + directions[di][1];
        }
        return ans;
    }
};
```

-

48.旋转矩阵

- $n \times n$ 的二维矩阵 `matrix` 表示一个图像。将图像顺时针旋转 90 度。不要使用另一个矩阵。



输入: matrix = [[1,2,3],[4,5,6],[7,8,9]]
输出: [[7,4,1],[8,5,2],[9,6,3]]

- 思路：先上下交换，再沿着对角线交换，就能得到最终元素了。

  8314c8f07fbc63bfeffea66756a8ca15

```
class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        int n = matrix.size();
        // 遍历矩阵，上下交换
        for(int i=0; i<n/2; i++){
            for(int j=0; j<n; j++){
                swap(matrix[i][j],matrix[n-1-i][j]);
            }
        }
        // 遍历矩阵，沿对角线交换(只用遍历下三角)
        for(int i=0;i<n;i++){
            for(int j=0;j<i;j++){
                swap(matrix[i][j],matrix[j][i]);
            }
```

```
        }
    }
};
```

-

240.搜索二维矩阵

- 搜索 `m x n` 矩阵 `matrix` 中的一个目标值 `target` 。

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

  - 每行的元素从左到右升序排列。
  - 每列的元素从上到下升序排列。
- 思路：对矩阵的每一行分别执行二分查找。

```
class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        for(auto& row : matrix){
            // 在有序范围 [begin, end) 中，查找第一个大于或等于 target 的元素位置
            auto it = lower_bound(row.begin(),row.end(),target);
            if( it!=row.end() && *it==target ){
                return true;
            }
        }
        return false;
    }
};
```

-